

scripts/04_run_scenarios.py

284 lines

```
1 """
2 04_run_scenarios.py
3 =====
4 Runs the three required demonstration scenarios (A: Startup to Target,
5 B: Target Tracking, C: Infeasible Target) closed-loop, using the fitted
6 `WellModel` inside the brute-force `ChokeMPC` controller acting on the noisy
7 `WellSimulator` plant.
8
9 For each scenario this script produces:
10 data/scenario_{X}_log.csv - full time series log
11 data/scenario_{X}_decisions.json - per-step MPC reasoning (for the dashboard)
12 figures/scenario_{X}.png - 6-panel results figure
13 and prints an explicit PASS/FAIL constraint-safety audit. `run_all.py`
14 propagates a non-zero exit code if any scenario fails its audit.
15 """
16
17 from __future__ import annotations
18
19 import json
20 import sys
21 from pathlib import Path
22
23 sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))
24
25 import numpy as np
26 import pandas as pd
27 import matplotlib.pyplot as plt
28
29 from config import DATA_DIR, FIG_DIR, LIMITS, CONTROLLER, SEED, SCENARIOS, TS_HOURS
30 from well_simulator import WellSimulator
31 from model import WellModel, OUTPUTS
32 from controller import ChokeMPC
33 from plotting import (
34     COLOR_CHOKE, COLOR_Q, COLOR_TARGET, COLOR_WHP, COLOR_FLP, COLOR_BHP,
35     style_axis, shade_safe_band, ramp_envelope, savefig,
36 )
37
38
39 def target_at(targets, t):
40     """Piecewise-constant target schedule: targets = [(start_h, value), ...]."""
41     val = targets[0][1]
42     for start_h, v in targets:
43         if t >= start_h:
44             val = v
45     return val
46
47
48 def run_scenario(key: str, model: WellModel) -> tuple[pd.DataFrame, list, dict]:
49     spec = SCENARIOS[key]
50     # NOTE: use a fixed, explicit per-scenario offset rather than hash(key).
51     # Python randomises string hashing per process (PYTHONHASHSEED), so
52     # hash(key) would silently give a different noise realisation on every
53     # run and the "reproducible with a fixed seed" guarantee would be false.
54     seed_offset = {"A": 1, "B": 2, "C": 3}[key]
55     sim = WellSimulator(seed=SEED + seed_offset, noise=True)
56
57     sim.reset()
58     controller = ChokeMPC(model=model)
59
60     u = spec["u_initial"]
61     rows = []
62     decisions = []
63
64     # Prime the plant to the initial choke position's own dynamics naturally
65     # by simply starting the loop; step 0 uses the reset (shut-in) measurement.
66     meas = {"Q": 0.0, "WHP": sim.history[-1]["WHP"], "FLP": sim.history[-1]["FLP"],
67            "BHP": sim.history[-1]["BHP"]}
```

```

68 n_steps = int(spec["duration_h"] / TS_HOURS)
69 for k in range(n_steps):
70     t = k * TS_HOURS
71     target = target_at(spec["targets"], t)
72
73     decision = controller.step(meas, u, target)
74     u = decision.u_selected
75
76     Q, WHP, FLP, BHP = sim.step(u)
77     meas = {"Q": Q, "WHP": WHP, "FLP": FLP, "BHP": BHP}
78
79     n_feas = sum(c.feasible for c in decision.candidates)
80     rows.append(
81     {
82         "time_h": t + TS_HOURS,
83         "target_Q": target,
84         "choke_pct": u,
85         "Q": Q, "WHP": WHP, "FLP": FLP, "BHP": BHP,
86         # Informational only - monitored as part of a complete
87         # production operating envelope, but NOT active constraints in
88         # this challenge and never used by the controller.
89         "WHT": sim.history[-1]["WHT"],
90         "AP": sim.history[-1]["AP"],
91         "n_candidates": len(decision.candidates),
92         "n_feasible": n_feas,
93         "all_infeasible": decision.all_infeasible,
94     }
95     )
96
97     chosen = decision.candidates[decision.selected_index]
98     decisions.append(
99     {
100         "t": t,
101         "u_prev": decision.u_previous,
102         "u_selected": decision.u_selected,
103         "target_Q": target,
104         "all_infeasible": decision.all_infeasible,
105         "selected_index": decision.selected_index,
106         "candidates": [
107         {
108             "u": round(c.u, 2),
109             "du": round(c.du, 2),
110             "feasible": c.feasible,
111             "reason": c.reject_reason,
112             "q_end": round(c.predicted_Q_end, 1),
113             "cost": round(c.cost, 2),
114         }
115         for c in decision.candidates
116         ],
117         "selected_trajectory": {
118             k2: [round(float(v), 1) for v in chosen.predicted_trajectory[k2]]
119             for k2 in OUTPUTS
120         },
121     }
122     )
123
124     df = pd.DataFrame(rows)
125     return df, decisions, spec
126
127
128 def audit(df: pd.DataFrame, key: str) -> bool:
129     """Explicit PASS/FAIL constraint-safety check against the TRUE hard limits
130     (not the controller's internal safety-margin limits)."""
131     L = LIMITS
132     checks = {
133         "WHP": (df["WHP"] >= L.whp_min).all() and (df["WHP"] <= L.whp_max).all(),
134         "FLP": (df["FLP"] >= L.flp_min).all() and (df["FLP"] <= L.flp_max).all(),
135         "BHP": (df["BHP"] >= L.bhp_min).all() and (df["BHP"] <= L.bhp_max).all(),
136         "ramp": (df["choke_pct"].diff().abs().dropna() <= CONTROLLER.max_move + 1e-6).all(),
137         "u_range": (df["choke_pct"] >= 0).all() and (df["choke_pct"] <= 100).all(),

```

```

138 "no_nan": not df.isna().any().any(),
139 }
140 passed = all(checks.values())
141 marks = " ".join(f"{k} {'OK' if v else 'FAIL'}" for k, v in checks.items())
142 print(f"[Scenario {key}] {marks} -> {'PASS' if passed else 'FAIL'}")
143 return passed
144
145
146 def plot_scenario(df: pd.DataFrame, spec: dict, key: str, path: Path) -> None:
147     fig, axes = plt.subplots(7, 1, figsize=(11, 17.5), sharex=True)
148     t = df["time_h"]
149     L = LIMITS
150
151     axes[0].plot(t, df["target_Q"], color=COLOR_TARGET, lw=1.6, ls="--", label="Target")
152     axes[0].plot(t, df["Q"], color=COLOR_Q, lw=2.0, label="Actual")
153     style_axis(axes[0], title=f"{spec['name']}: Oil Flow Rate", ylabel="Q [bbl/hr]")
154     axes[0].legend(loc="best", fontsize=8)
155
156     axes[1].plot(t, df["WHP"], color=COLOR_WHP, lw=1.8)
157     shade_safe_band(axes[1], L.whp_min, L.whp_max, CONTROLLER.margin_whp, CONTROLLER.margin_whp)
158     style_axis(axes[1], title="Wellhead Pressure", ylabel="WHP [psi]")
159
160     axes[2].plot(t, df["FLP"], color=COLOR_FLP, lw=1.8)
161     shade_safe_band(axes[2], L.flp_min, L.flp_max, CONTROLLER.margin_flp, CONTROLLER.margin_flp)
162     style_axis(axes[2], title="Flowline Pressure", ylabel="FLP [psi]")
163
164     axes[3].plot(t, df["BHP"], color=COLOR_BHP, lw=1.8)
165     shade_safe_band(axes[3], L.bhp_min, L.bhp_max, CONTROLLER.margin_bhp, CONTROLLER.margin_bhp)
166     style_axis(axes[3], title="Bottom Hole Pressure", ylabel="BHP [psi]")
167
168     axes[4].plot(t, df["choke_pct"], color=COLOR_CHOKE, lw=2.0, drawstyle="steps-post")
169     axes[4].set_ylim(-5, 105)
170     style_axis(axes[4], title="Choke Position (5%/step ramp limit)", ylabel="Choke [%]")
171
172     axes[5].plot(t, df["n_feasible"], color="#7c3aed", lw=1.6)
173     axes[5].axhline(0, color="#dc2626", lw=1, ls=":")
174     style_axis(axes[5], title="Feasible Candidates per Step (of 21)",
175     ylabel="# feasible")
176
177     # Informational variables (WHT, AP): monitored as part of a complete
178     # operating envelope per the problem statement, but not active constraints
179     # and never used by the controller.
180     ax6 = axes[6]
181     ax6.plot(t, df["WHT"], color="#c2410c", lw=1.6, label="WHT [degF]")
182     ax6.set_ylabel("WHT [degF]")
183     ax6b = ax6.twinx()
184     ax6b.plot(t, df["AP"], color="#0891b2", lw=1.4, ls="--", label="AP [psi]")
185     ax6b.set_ylabel("AP [psi]")
186     lines = ax6.get_lines() + ax6b.get_lines()
187     ax6.legend(lines, [l.get_label() for l in lines], loc="best", fontsize=8)
188     style_axis(ax6, title="Informational only - not active constraints (WHT, AP)",
189     xlabel="Time [h]")
190
191     fig.tight_layout()
192     savefig(fig, path)
193     print(f" wrote {path}")
194
195
196 def plot_compact_summary(logs: dict) -> None:
197     """A single landscape figure with all three scenarios side by side.
198
199     The per-scenario 7-panel figures are portrait and far too tall to embed in
200     a slide; this is the wide, at-a-glance version for the deck and the report
201     summary. Rows: oil rate vs target, BHP against its limit, choke position.
202     """
203     fig, axes = plt.subplots(3, 3, figsize=(15, 7.2), sharex="col")
204     L = LIMITS
205     for col, key in enumerate(("A", "B", "C")):
206         df = logs[key]
207         t = df["time_h"]

```

```

208 spec = SCENARIOS[key]
209
210 ax = axes[0][col]
211 ax.plot(t, df["target_Q"], color=COLOR_TARGET, lw=1.6, ls="--", label="Target")
212 ax.plot(t, df["Q"], color=COLOR_Q, lw=2.0, label="Actual")
213 style_axis(ax, title=spec["name"], ylabel="Q [bbl/hr]" if col == 0 else None)
214 ax.legend(loc="lower right", fontsize=7)
215
216 ax = axes[1][col]
217 ax.plot(t, df["BHP"], color=COLOR_BHP, lw=1.8)
218 shade_safe_band(ax, L.bhp_min, L.bhp_max, CONTROLLER.margin_bhp, CONTROLLER.margin_bhp)
219 ax.set_ylim(L.bhp_min - 60, max(df["BHP"].max() + 60, L.bhp_min + 200))
220 style_axis(ax, title=f"BHP (min {df['BHP'].min():.0f} psi, limit {L.bhp_min:.0f})",
221 ylabel="BHP [psi]" if col == 0 else None)
222
223 ax = axes[2][col]
224 ax.plot(t, df["choke_pct"], color=COLOR_CHOKE, lw=2.0, drawstyle="steps-post")
225 ax.set_ylim(-5, 105)
226 style_axis(ax, title="Choke position", xlabel="Time [h]",
227 ylabel="Choke [%]" if col == 0 else None)
228
229 fig.suptitle("Scenario results - target tracking and constraint compliance",
230 fontweight="bold")
231 fig.tight_layout()
232 savefig(fig, FIG_DIR / "scenarios_compact.png")
233 print(f" wrote {FIG_DIR / 'scenarios_compact.png'}")
234
235
236 def main():
237     model = WellModel.load(DATA_DIR / "fitted_model.json")
238
239     all_pass = True
240     summary = {}
241     logs = {}
242     for key in ("A", "B", "C"):
243         print(f"\nRunning {SCENARIOS[key]['name']} ...")
244         df, decisions, spec = run_scenario(key, model)
245
246         df.to_csv(DATA_DIR / f"scenario_{key}_log.csv", index=False)
247         (DATA_DIR / f"scenario_{key}_decisions.json").write_text(json.dumps(decisions))
248
249         logs[key] = df
250         ok = audit(df, key)
251         all_pass = all_pass and ok
252
253     plot_scenario(df, spec, key, FIG_DIR / f"scenario_{key}.png")
254
255     summary[key] = {
256         "name": spec["name"],
257         "pass": ok,
258         # A single last sample carries a full dose of sensor noise, so the
259         # settled mean over the final 20 intervals is the honest number to
260         # quote for tracking performance.
261         "final_Q": float(df["Q"].iloc[-1]),
262         "settled_Q": float(df["Q"].tail(20).mean()),
263         "settled_Q_std": float(df["Q"].tail(20).std()),
264         "final_target": float(df["target_Q"].iloc[-1]),
265         "final_choke": float(df["choke_pct"].iloc[-1]),
266         "min_whp": float(df["WHP"].min()), "max_whp": float(df["WHP"].max()),
267         "min_flp": float(df["FLP"].min()), "max_flp": float(df["FLP"].max()),
268         "min_bhp": float(df["BHP"].min()), "max_bhp": float(df["BHP"].max()),
269     }
270
271     plot_compact_summary(logs)
272
273     (DATA_DIR / "scenario_summary.json").write_text(json.dumps(summary, indent=2))
274     print("\n" + "=" * 60)
275     print("ALL SCENARIOS:", "PASS" if all_pass else "FAIL")

```

```
276 print("=" * 60)
277
278 if not all_pass:
279     sys.exit(1)
280
281
282 if __name__ == "__main__":
283     main()
284
```